

Submission Link:

<https://forms.office.com/Pages/ResponsePage.aspx?id=jaCKheGUxkuc84wRtTBwHANSILeDKUNOi1YB2VJD6t1UMk5TVjZYVEZIN0tTTkxKNkNVUkIRQ0IDMC4u>

Answer all the questions. Mark the correct question number. Show calculations and diagrams wherever necessary. Q1 carries 10 marks, Q2 and Q3 carry 8 marks and Q4 carries 4 marks. [Max: 30 marks, Duration 90 minutes.]

1. An embedded system unit has a processor, a 64 KB cache memory and a 1 MB main memory. The word length is 64 bits. The cache memory is direct mapped with a block size of 256 words. Operating system kernel code is stored from word 0 to word 4095 (in decimal) in the main memory. The kernel code is in an infinite loop from word 2 to word 4092 (both inclusive). Execution of the code within this loop is fully sequential without any branch or loop instructions except two Interrupt Service Routine (ISR) calls. These ISR calls are at word 2000 (ISR_A) and 4000 (ISR_B). Both these ISRs are 1024 words each and are loaded from the word addresses 8192 and 10240, respectively. The last line of each of the ISR is a return statement back to the kernel. Each ISR code also is fully sequential without any branch or loop instructions. Assume caches are initially empty. All data of kernel and ISRs are in processor registers and will not affect cache mapping.
 - (a) Show the physical address split up of tag, set-index and byte offset.
 - (b) How many cache sets will be still empty after 50 iterations of the kernel loop? List them.
 - (c) If each context switch (control transfer from kernel to ISR/vice versa) takes ‘a’ milliseconds, execution of an instruction word takes ‘b’ milliseconds if it is a cache hit or ‘c’ milliseconds if it is a cache miss, then what is the execution time of 10 iterations of the kernel loop?
2. Consider a 4-way set associative cache that uses pseudo LRU block replacement policy. Assume all the cache blocks are initially empty and filling up of empty blocks in a given cache set happens from way-0 to way-3. Consider the following 13 block numbers all mapped to set n given in the order of arrival. A, R, S, R, B, A, D, C, S, A, S, B, E.
 - (a) Find the pseudo LRU subtree after processing these 13 blocks.
 - (b) How many conflict misses are there?
 - (c) Which of the 4 ways (way-0 to way-3) has maximum cache block replacement during processing these requests?
3. Consider a dynamically scheduled single-issue pipeline with speculation using a branch predictor that always predicts the branch as taken. ROB has a capacity of 6 entry. The pipeline has one floating point unit, one memory unit (L.D /S.D), and one integer unit. Branch is also scheduled in the integer unit and branch outcome is written to CDB to initiate committing in ROB. The floating-point unit takes 4 cycles for multiplication and 2 cycles for addition and they are fully pipelined with normal operand forwarding. Even though the floating-point unit is capable of doing two operations, it can initiate only one operation in a given clock cycle. The integer unit

takes 1 cycle and supports full operand forwarding. Memory access of a load operation is done in the immediate cycle after address computation. Stores access memory only upon commit. Assume you have 2 reservation station entries for each functional unit. At most one CDB write can happen in a cycle.

```

      L.D    F1, 8 (R0)
loop: L.D    F2, 0 (Rx)
      MUL.D  F3, F2, F1
      ADD.D  F5, F5, F3
      S.D    F5, 0 (Ry)
      SUB    Ry, Ry, #8
      SUB    Rx, Rx, #8
      BNEZ   Rx, loop
  
```

Fill up the location of operand, time of issue, execution, MEM access, CDB write and commit for 2 iterations of the loop for the code given above. (Use the following table format)

Iter No	Instruction	Operand 1	Operand 2	Issue	Execute	MEM Access	CDB Write	Commit
1	L.D F1, 8 (R0)	Imm	RF	1	2	3	4	5
1	L.D F2, 0 (Rx)	Imm	RF	2	3	4	5	6
1	MUL.D F3, F2, F1	CDB	CDB	3				

4. Consider a cache that has a miss penalty of 200 clock cycles, and all instructions normally take 1 clock cycles for a perfect cache that is 100% hit. It was found that the processor with the perfect cache has 7 times speed up over the realistic cache while running a program that has an average of 1.5 memory references per instruction.
- What is the miss rate of the realistic cache and average MPKI for this program?
 - How much slowdown (approximate) will the program experiences if there was no cache memory in the memory hierarchy?